

As for the length of each indentation level, the common practice in Python code is four spaces per level of indentation. The space characters in the following example have been marked with periods to make them visible:

```
def getCatAmount():  
    ....numCats = input('How many cats do you have?')  
  
    ....if int(numCats) < 6:  
  
        .....print('You should get more cats.')
```

The four-space standard has practical benefits compared to the alternatives; using eight spaces per level of indentation results in code that quickly runs up against line length limits, whereas using two space characters per level of indentation can make the differences in indentation hard to see. Programmers often don't consider other amounts, such as three or six spaces, because they, and binary computing in general, have a bias for numbers that are powers of two: 2, 4, 8, 16, and so on.

Spacing Within a Line

Horizontal spacing has more to it than just the indentation. Spaces are important for making different parts of a code line appear visually distinct. If you never use space characters, your line can end up dense and hard to parse. The following subsections provide some spacing rules to follow.

Put a Single Space Between Operators and Identifiers

If you don't leave spaces between operators and identifiers, your code will appear to run together. For example, this line has spaces separating operators and variables:

```
YES: blanks = blanks[:i] + secretWord[i] + blanks[i + 1 :]
```

This line removes all spacing:

```
NO:  blanks=blanks[:i]+secretWord[i]+blanks[i+1:]
```
